АиСД у2023. Первый семестр

Домашние задания М3132-М3133

⟨Версия от 6 сентября 2023 г.⟩

Темы

1 Введение. Асимптотические оценки алгоритмов. 1

1.1 Конспект от Гриши @kgrigoriy Хлытина 2

1.2 ДЗ 6

Неделя 1. Введение. Асимптотические оценки алгоритмов.

Конспект от Гриши @kgrigoriy Хлытина

```
/* Функция min_search поиска минимума в массиве arr */
int min_search(int[] arr) {
    /* Инициализируем переменную ans максимальным числом типа int */
    int ans = Integer.MAX_VALUE;
    /* Пройдем циклом for по всему массиву */
    for (int i = 0; i < arr.length; i++) {
        /* Если текущий элемент меньше текущего минимума */
        if (arr[i] < ans) {
            ans = arr[i];
        }
    }
    /* Вернем найденный минимум */
    return ans;
}
```

Listing 1: Поиск минимума в массиве

```
/* Рекурсивная функция sum */
/* Подсчитывает сумму всех натуральных чисел от 0 до x */
/* BigInteger - тип данных для больших целых чисел (без ограничения размера) */
BigInteger sum(BigInteger x) {
    /* Если x <= 0, то вернем как ответ 0 */
    if (x <= 0) {
        /* Вернем BigInteger со значением равным нулю */
        return BigInteger("0");
    } else {
        /* Иначе рекурсивно вычислим сумму от 0 до (x - 1) */
        BigInteger ans = sum(x - 1);
        /* Вычислим ответ от 0 до x*/
        ans = ans + x;
        /* Вернем результат */
        return ans;
    }
}
```

Listing 2: Сумма всех натуральных чисел от 0 до x

Конспект лекции:

- ▷ Что такое алгоритм?

- Это **конечная** последовательность действий для абстрактного вычислителя;
- Алгоритм получает на вход данные *input*, и после завершения работы выдает данные *output*;

▷ Абстрактный вычислитель: Машина Тьюринга, RAM-модель. Какие отличия между RAM-моделью и реальным компьютером?

- В RAM-модели программы (набор инструкций) хранятся в отдельной памяти, а в реальном компьютере хранятся вместе с другими данными в оперативной памяти;
- В RAM-модели мы грубо считаем, что каждая инструкция выполняется за **одну** условную единицу времени, в то время как для реального компьютера это не так (см. пункт ниже);

▷ Реальный компьютер: язык ассемблера, кэш-пямять.

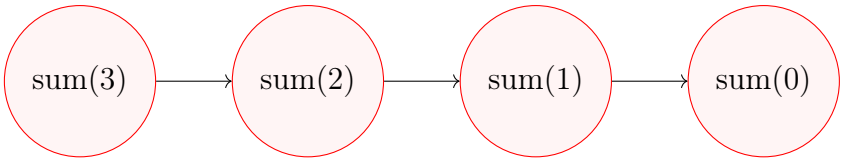
- Подробнее про язык ассемблера можно почитать здесь (не обязательно);
- Арифметические инструкции (например `ADD`) работают примерно за **один** такт процессора, в то время как инструкции работы с оперативной памятью (например `MOV`) работают за несколько сотен тактов;
- Исходя из пункта выше, для оптимизации процессор *кеширует* данные:
  То есть при обращении в оперативную память к ячейке с адресом i, процессор заодно выгружает из оперативной памяти несколько соседних с i-ой ячеек (это называется *кеш-линией*) и запоминает их в *кеш-память* (которая очень быстрая и находится прямо в процессоре).
  Теперь, если мы захотим обратится в оперативную память к ячейке с адресом $i-1$, процессор не пойдет в оперативную память, а возьмет данные из *кеш*-а (и это будет в несколько сотен раз быстрее, примерно за **один** такт процессора);

▷ Как оценить производительность алгоритма?

- Засекать время плохо — у разных процессоров разная производительность;
- Будем считать функцию $T(n)$ — количество абстрактных операций (арифметические операции и операции обращения к памяти) в RAM-модели, которые делает наш алгоритм, если на вход алгоритму подаются входные данные *input* и размер входных данных равен $n = |input|$ (в простейшем случае n это количество чисел подающихся на вход нашему алгоритму);
- Псевдокод алгоритма на языке `Java` для поиска минимума в массиве, который был рассмотрен на лекции, приведен на листинге 1;
- На лекции мы посчитали, что *время работы* такого алгоритма с листинга 1 примерно равно $T(n) = 2 + 3 \cdot n$ абстрактных операций (по одной абстрактной операции на строчках: `ans = Integer.MAX_VALUE`, `return ans`; а так же цикл `for` размера n, на каждой итерации которого выполняется около трех абстрактных операций);

▷ А как оценить производительность алгоритма, если он представляет из себя рекурсивную функцию?

- Построим **дерево рекурсивных вызовов**!
- Посмотрим на листинг 2, на котором изображена рекурсивная функция $sum(x)$, которая вычисляет сумму всех натуральных чисел от 0 до x;
- Как оценить её *время работы*? Как для неё будет выглядеть функция $T(x)$ количества абстрактных операций в зависимости от числа x? А как для неё будет выглядеть функция $T(n)$, где n — размер входа?
  (Обратите внимание: в данном случае, так как число x подающееся на вход алгоритму может быть очень большим по значению, размер входа будет равен $n = log(x)$, так как размер входа это количество цифр в записи числа x)
- Построим *дерево рекурсивных вызовов* например для вызова функции $sum(3)$, как делали это на лекции:



- Сколько абстрактных операций делает функция sum на одном слое в *дереве рекурсивных вызовов*?
  По её коду видно, что на одном слое она только складывает два числа: `ans = ans + x`. Сейчас для простоты **временно будем считать**, что **суммирование** двух больших чисел работает за **одну** абстрактную операцию (хотя на самом деле такое суммирование работает за количество цифр в записи числа — то есть за $\mathcal{O}(log\, x)$);
- На рисунке выше видно, что количество слоёв в нашем *дереве рекурсивных вызовов* равно x, причем мы сказали что на каждом слое функция sum делает только одну абстрактную операцию, таким образом время работы функции $sum(x)$ будет равно *количество слоев* $\times$ *количество операций на слое* и будет равно: $T(x) = x \cdot 1 = \Theta(x)$.
  Если расписать $T(x)$ рекурсивно, то будет: $T(x) = T(x-1)+1$ (так как мы сказали что на каждом слое функция sum делает **одну** операцию, а так же рекурсивно вызывает саму себя с аргументом ровно на один меньше);
- **Важно для понимания №1**:
  если бы наша исследуемая функция sum на каждом слое делала не 1 операцию, а x операций, тогда ее время работы было бы: $T(x) = T(x-1)+x$, и было бы верно, что $T(x) = \Theta(x^2)$, так как количество слоев все еще было бы равно x и на каждом слое функция делала бы порядка x операций: $T(x) = x + (x-1) + (x-2) + (x-3) + (x-4) + \ldots + 2 + 1 = \Theta(x^2)$;
- **Важно для понимания №2**:
  так как мы уже выше сказали, что суммирование двух больших чисел `ans = ans + x` на самом деле работает не за одну операцию, а за количество цифр в записи числа — то есть за $\mathcal{O}(log(x))$, получается что на одном слое функция sum на самом деле делает порядка $log(x)$ операций, и на самом деле время ее работы: $T(x) = T(x-1) + log(x)$. Количество слоев все еще равно x, на каждом слое функция sum делает порядка $log(x)$ операций, получаем: $T(x) = log(x) + log(x-1) + log(x-2) + \ldots + log(2) + log(1) = \Theta(x \cdot log(x))$;

Def. Так как нас интересует только *асимптотика* функции $T(n)$ (порядок роста функции $T(n)$ относительно размера n входных данных), мы сказали, что нам интересен только наибольший член этой функции, а так же неинтересны константы. Чтобы формально и коротко это записать, мы ввели следующие обозначения:

- $\star$ $T(n) = \mathcal{O}(f(n))$
- $\star$ $T(n) = \Omega(g(n))$
- $\star$ $T(n) = \Theta(h(n))$

В примерах выше, функции $f(n)$, $g(n)$ — это соответственно *оценка сверху* и *оценка снизу* для нашей функции $T(n)$.

Def. Напомним используемые далее определения:

- $\star$ $f(n) = \mathcal{O}(g(n)) \equiv \exists n_0, c > 0 : \forall n \geqslant n_0 : f(n) \leqslant c \cdot g(n)$
- $\star$ $f(n) = \Omega(g(n)) \equiv \exists n_0, c > 0 : \forall n \geqslant n_0 : f(n) \geqslant c \cdot g(n)$
- $\star$ $f(n) = \Theta(g(n)) \equiv \exists n_0, c_1, c_2 > 0 : \forall n \geqslant n_0 :$
$$c_1 \cdot g(n) \leqslant f(n) \leqslant c_2 \cdot g(n)$$

Def. Когда мы записываем $T(n) = T(n-1) + f(n)$, это значит, что при построении *дерева рекурсивных вызовов* количество слоёв будет равно n и **ветвления не будет** (дерево будет выглядеть как прямая линия), на каждом слое функция будет делать: $f(n)$, $f(n-1)$, $f(n-2)$, $\ldots$, $f(2)$, $f(1)$ операций соответственно, тогда для неё верно:

- $\star$ $T(n) = f(n) + f(n-1) + f(n-2) + \ldots + f(2) + f(1)$
- $\star$ Если $f(n) = n$, получаем $T(n) = \Theta(n^2)$
- $\star$ Если $f(n) = log(n)$, получаем $T(n) = \Theta(n \cdot log(n))$
- $\star$ Если $f(n) = 1$, получаем $T(n) = \Theta(n)$

Доказательство этих фактов остается читателю в качестве упражнения.

Def. Когда мы записываем $T(n) = 2 \cdot T(n-1) + f(n)$, это значит, что при построении *дерева рекурсивных вызовов* количество слоёв будет равно n и **ветвление будет** (дерево будет выглядеть как полное двоичное дерево), на каждом слое функция **в сумме по всему слою** будет делать: $f(n)$, $2 \cdot f(n-1)$, $4 \cdot f(n-2)$, $\ldots$, $2^n \cdot f(1)$ операций соответственно, тогда для неё верно:

- $\star$ $T(n) = f(n) + 2 \cdot f(n-1) + 4 \cdot f(n-2) + \ldots + 2^n \cdot f(1)$

Def. Когда мы записываем $T(n) = T(n/2) + f(n)$, это значит, что при построении *дерева рекурсивных вызовов* количество слоёв будет равно $log(n)$ и **ветвления не будет** (дерево будет выглядеть как прямая линия), на каждом слое функция будет делать: $f(n)$, $f(n/2)$, $f(n/4)$, $\ldots$, $f(1)$ операций соответственно,

> тогда для неё верно:
>
> $\star$ $T(n) = f(n) + f(n/2) + f(n/4) + \ldots + f(1)$

ДЗ

Докажите по определению — выберите константы c и n_0 и объясните соответствующее неравенство

Докажите или опровергните следующие утверждения. Для опровержения достаточно привести пример, на котором утверждение неверно. Все функции положительные.

1.1. Если $f(n) = \mathcal{O}(g(n))$, то $g(n) = \mathcal{O}(f(n))$

1.2. Если $f(n) = \mathcal{O}(g(n))$, то $2^{f(n)} = \mathcal{O}(2^{g(n)})$

1.3. $f(n) = \mathcal{O}(f(n)^2)$

1.4. $f(n) = \Theta(f(\frac{n}{2}))$

1.5. Если $f(n) = \mathcal{O}(g(n))$, то $g(n) = \Omega(f(n))$

Докажите по определению

1.6. Докажите по определению, что если $f_1(n) = \mathcal{O}(g_1(n))$ и $f_2(n) = \mathcal{O}(g_2(n))$, то $f_1(n) + f_2(n) = \mathcal{O}(g_1(n) + g_2(n))$.

1.7. Докажите по определению, что $\max(f(n), g(n)) = \Theta(f(n) + g(n))$, если $f(n)$ и $g(n)$ неотрицательны.

1.8. Докажите по определению, что $\sum\limits_{i=1}^{n+5} 2^i = \mathcal{O}(2^n)$.

1.9. Докажите по определению, что $\frac{n^3}{6} - 7n^2 = \Omega(n^3)$

1.10. Докажите по определению, что $\log_3(n^3 + 5n + 10) = \Theta(\log(n))$

1.11. Докажите по определению, что $2^n \cdot n^5 = \mathcal{O}(2.1^n)$

Пусть $p(n) = \sum\limits_{i=0}^{d} a_i n^i$, где $a_d > 0$ — полином степени d от n. Используя определения, полагая, что k — константа, докажите, что:

1.12. Если $k \geq d$, то $p(n) = \mathcal{O}(n^k)$

1.13. Если $k = d$, то $p(n) = \Theta(n^k)$

1.14. Если $k \leq d$, то $p(n) = \Omega(n^k)$

Следующий тип заданий:

1.15. Придумайте две положительные функции $f(n)$ и $g(n)$, такие что не выполнено ни одно из двух: $f(n) = \mathcal{O}(g(n))$ и $f(n) = \Omega(g(n))$.

1.16. Докажем, что $n^2 = \mathcal{O}(n)$ по индукции. Для этого докажем, что для любого k верно, что $kn = \mathcal{O}(n)$. Действительно, для $k = 1$ это верно. Пусть утверждение верно для $k-1$, тогда $kn = (k-1)n + n = \mathcal{O}(n)$. Таким образом, это верно для всех k, а значит и для $k = n$, таким образом $n \cdot n = \mathcal{O}(n)$. Где ошибка в этом доказательстве?

1.17. Докажите или опровергните, что $\log(n!) = \Theta(n \log n)$.

1.18. Докажите, что $\sum\limits_{i=1}^{n} \frac{n}{i} = \mathcal{O}(n \log n)$.

1.19. Для каждой из приведенных ниже программ найдите и аргументируйте точную $\mathcal{O}$-асимптотику времени ее работы.

(a)
```
for i = 0..n:
    for j = 0..i:
        for k = 0..j:
            print(i, j, k)
```

(b)
```
for i = 0..n:
    j = 0, k = 2 * i
    while j < k:
        j++, k--
```

(c)
```
i = 1
while i < n:
    for j = 0..i:
        print(i, j)
    i = i * 2
```

(d)
```
for i = 0..n:
    j = i
    while j > 0:
        j = j / 2
```

Время работы некоторого алгоритма задано следующим рекуррентным соотношением. Найдите Θ-асимптотику времени работы этого алгоритма, *построив дерево рекурсивных вызовов.*

1.20. $T(n) = T(n-1) + n$

1.21. $T(n) = 3T\left(\frac{n}{2}\right) + 3$

1.22. $T(n) = T\left(\frac{n}{3}\right) + \log n$